MECANISMOS DE SINCRONIZAÇÃO

FPPD — Unidade 3

2026

ROTEIRO

1. **Semáforos**
   Dijkstra, P/V, tipos e soluções com semáforos
2. **Monitores**
   Encapsulamento, cond vars e soluções com monitores
3. **Canais para Sincronização**
   Comunicação como mecanismo de sincronização
4. **Comparação**
   Quando usar cada mecanismo

SEMÁFOROS

Mecanismo clássico de Dijkstra

O QUE É UM SEMÁFORO?

> Proposto por **Dijkstra (1965)** para problemas de sincronização em sistemas concorrentes.

- Estrutura de dados com um **contador** e uma **fila** de identificadores de processos (PIDs)
- O contador limita **quantos processos** podem prosseguir (por exemplo, entrar em região protegida)
- Duas operações atômicas: **Wait** (`P`, `down`) e **Signal** (`V`, `up`)

OPERAÇÕES: WAIT E SIGNAL

Dijkstra usou as iniciais em holandês: **P** de *probeer* (tentar) e **V** de *verhoog* (incrementar). Na literatura em inglês: **Wait/Down** e **Signal/Up**.

P — WAIT / DOWN

- Se **contador > 0**: decrementa e o processo continua
- Se **contador == 0**: bloqueia e enfileira o PID

V — SIGNAL / UP

- Se a **fila não está vazia**: desbloqueia um processo
- Senão: **incrementa** o contador

As operações são **atômicas**: não podem ser interrompidas por interleaving de outros processos no meio da atualização.

TIPOS DE SEMÁFOROS

BINÁRIO

- Valores **0 ou 1**
- Inicialização típica: **1** para exclusão mútua
- Equivale a um **mutex** quando usado só para lock/unlock

CONTADOR (GERAL)

- Qualquer valor **não negativo**
- Inicialização **N** para permitir até N acessos simultâneos
- Útil para **pools** de recursos e coordenação de produtor-consumidor

SEMÁFOROS FORTES VS. FRACOS

FRACO (WEAK)

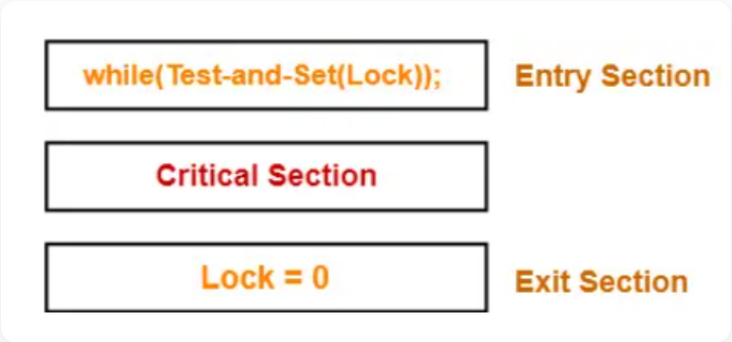
- Processo desbloqueado escolhido de forma **arbitrária** na fila
- Maior liberdade ao escalonador — pode aumentar concorrência aparente
- Risco de **starvation** se não houver justiça adicional

FORTE (STRONG)

- Ordem típica **FIFO** na fila de bloqueados
- Comportamento mais previsível; evita starvation por ordem de chegada
- Pode reduzir paralelismo útil em alguns cenários

IMPLEMENTAÇÃO DE SEMÁFOROS

- O contador do semáforo é variável compartilhada: sem primitivas corretas, haveria **data race**
- **Test-and-Set Lock (TSL)**: instrução de hardware que lê um endereço e escreve valor (ex.: 1) **atomicamente**
- Permite construir exclusão mútua e, a partir dela, implementar filas e contadores com segurança
- Detalhes de microarquitetura e políticas do SO costumam aparecer na disciplina de **Sistemas Operacionais**



SEÇÃO CRÍTICA COM SEMÁFOROS

Semáforo binário `S` inicializado em **1**: `wait(S)` adquire exclusão; `signal(S)` libera.

Algorithm 6.1: Critical section with semaphores (two processes)	
binary semaphore S ← (1, ∅)	
p	**q**
loop forever	loop forever
p1: non-critical section	q1: non-critical section
p2: wait(S)	q2: wait(S)
p3: critical section	q3: critical section
p4: signal(S)	q4: signal(S)

Ben-Ari, Algorithm 6.1

> Invariante: o valor do semáforo indica quantos processos podem entrar — com valor inicial 1, no máximo um.

PRODUTOR-CONSUMIDOR COM SEMÁFOROS

- Dois **semáforos contadores**: `notFull` (init `N`, vagas) e `notEmpty` (init `0`, itens)
- Produtor e consumidor são **processos** independentes

Algorithm 6.8: Producer-consumer (finite buffer, semaphores)

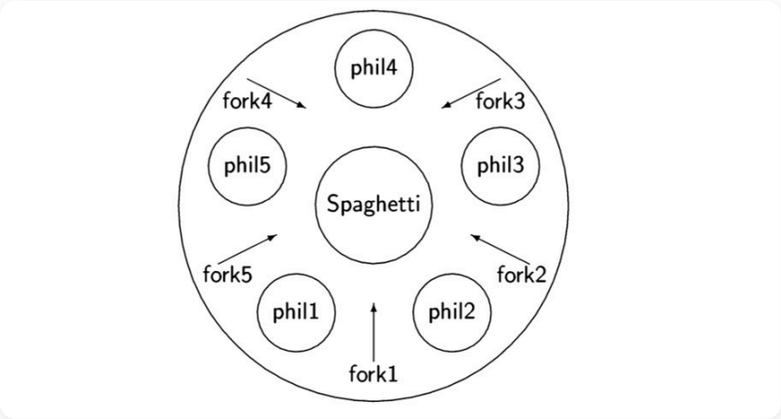
```
finite queue of dataType buffer ← empty queue
semaphore notEmpty ← (0, ∅)
semaphore notFull ← (N, ∅)
```

	producer		**consumer**
	dataType d		dataType d
	loop forever		loop forever
p1:	d ← produce	q1:	wait(notEmpty)
p2:	wait(notFull)	q2:	d ← take(buffer)
p3:	append(d, buffer)	q3:	signal(notFull)
p4:	signal(notEmpty)	q4:	consume(d)

Ben-Ari, Algorithm 6.8

FILÓSOFOS COM SEMÁFOROS

- Cada garfo é um **semáforo binário** (`fork[i]` init 1)
- Cada filósofo é um **processo**
- Filósofo `i` adquire `fork[i]` e `fork[i+1]` antes de comer



Algorithm 6.10: Dining philosophers (first attempt)

semaphore array [0..4] fork ← [1,1,1,1,1]

```
    loop forever
p1:     think
p2:     wait(fork[i])
p3:     wait(fork[i+1])
p4:     eat
p5:     signal(fork[i])
p6:     signal(fork[i+1])
```

Ben-Ari, Algorithm 6.10

Se todos pegam o garfo à esquerda primeiro → espera circular → deadlock.

CONDIÇÕES DE COFFMAN

Deadlock exige que **quatro** condições simultâneas sejam satisfeitas; quebrar *qualquer uma* elimina a possibilidade.

1. Exclusão mútua — recursos não compartilháveis

2. Hold and wait — processo retém recurso enquanto pede outro

3. Não preempção — recursos só liberados voluntariamente

4. Espera circular — cadeia de processos esperando uns pelos outros

SOLUÇÕES PARA OS FILÓSOFOS

QUEBRAR A SIMETRIA

- Um filósofo pega garfos na **ordem inversa**
- Ou: ordenar globalmente e adquirir do **menor para o maior**
- Elimina a **espera circular** (condição 4)

LIMITAR PARTICIPANTES

- Semáforo contador `seats = N-1`
- No máximo `N-1` filósofos tentam pegar garfos
- Elimina a **hold and wait** (condição 2)

Ambas as estratégias quebram uma das condições de Coffman — generalizáveis a outros cenários de alocação de recursos.

SEMÁFOROS EM GO

A linguagem **não expõe** um tipo `semaphore` na biblioteca padrão; o estilo idiomático privilegia **canais** e `sync.Mutex`.

- `sync.Mutex` comporta-se como **semáforo binário** para exclusão mútua
- Semáforo contador pode ser **simulado** com canal com buffer de tamanho `N`

```go
sem := make(chan struct{}, N)
sem <- struct{}{}
<-sem
```

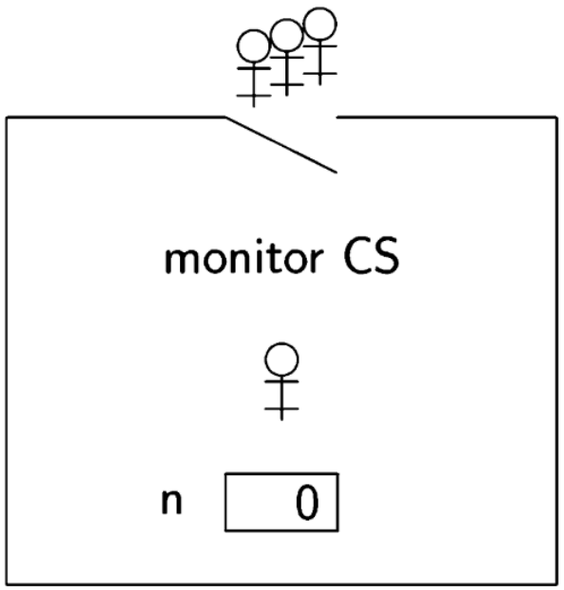
Cada token no buffer representa uma permissão; envio e recebimento sincronizam como P/V.

MONITORES

Sincronização estruturada e orientada a módulos

O QUE É UM MONITOR?

- Formalizado por **Hoare (1974)** e **Brinch Hansen (1975)**
- Módulo que encapsula dados e procedimentos com **exclusão mútua automática** — apenas uma thread ativa dentro do monitor por vez
- Generalização em espírito **orientado a objetos** — dados privados, métodos como únicos pontos de acesso
- Presente em **Ada**, **Java** (`synchronized`), **C#** (`lock`)



ATOMICIDADE DAS OPERAÇÕES DO MONITOR

Dois processos **p** e **q** chamam `CS.increment`. Embora a operação contenha múltiplas instruções (`temp ← n; n ← temp+1`), o monitor garante que **apenas um** executa por vez.

> O programador não precisa adicionar locks manualmente — a exclusão mútua é **automática**.

Algorithm 7.1: Atomicity of monitor operations

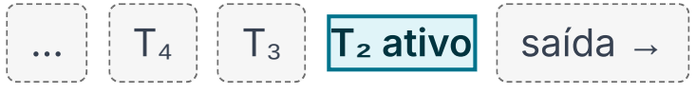
```
monitor CS
   integer n ← 0

   operation increment
      integer temp
      temp ← n
      n ← temp + 1
```

p	q
p1: CS.increment	q1: CS.increment

Ben-Ari, Algorithm 7.1

MONITOR — FILAS DE ENTRADA E DE CONDIÇÃO

Threads que chamam métodos do monitor competem pela exclusão: uma **ativa**, demais na **fila de entrada**.



Somente a thread ativa executa corpo de método; ao sair, o runtime escolhe a próxima da fila de entrada.

O monitor possui **duas categorias de filas**: a **fila de entrada** (threads que querem entrar) e as **filas de condição** (threads que já estavam dentro, mas chamaram `wait()` e aguardam uma condição).

SEÇÃO CRÍTICA COM MONITORES

> O monitor resolve a seção crítica **por construção**: apenas uma thread ativa dentro do monitor por vez — exclusão mútua implícita.

- O compilador ou runtime insere os locks automaticamente — o programador **não precisa** gerenciá-los
- Comparado com semáforos: menos propenso a erros (esquecer `signal`, inverter ordem)
- Para **apenas** exclusão mútua, não são necessárias variáveis de condição

VARIÁVEIS DE CONDIÇÃO

Permitem sincronização **além** da exclusão mútua: uma thread espera até que um predicado (condição lógica) se torne verdadeiro.

- `waitC(cond)`: bloqueia a thread, **libera o lock** do monitor e a coloca na fila da condição
- `signalC(cond)`: acorda **uma** thread esperando nessa variável
- `broadcast()` / `signalAll()`: acorda **todas** — usar quando múltiplas threads podem prosseguir (ex.: buffer esvaziou e há vários produtores esperando)

> Ben-Ari usa `waitC`/`signalC` (C de *Condition*) para distinguir das operações `wait`/`signal` de semáforos. Em Java: `wait()`/`notify()`. Em Go: `cond.Wait()`/`cond.Signal()`.

VARIÁVEIS DE CONDIÇÃO EM GO: `sync.Cond`

Go não tem monitor como palavra-chave, mas `sync.Cond` implementa variáveis de condição associadas a um `sync.Mutex`.

CRIAÇÃO

```go
var mu sync.Mutex
cond := sync.NewCond(&mu)
```

CORRESPONDÊNCIA

Ben-Ari	Go
`waitC(c)`	`cond.Wait()`
`signalC(c)`	`cond.Signal()`
`broadcast`	`cond.Broadcast()`

PADRÃO DE USO

```go
mu.Lock()
for !condicao {
    cond.Wait() // libera mu e espera
}
// condição satisfeita
mu.Unlock()
```

`Wait()` **libera o lock** automaticamente ao bloquear e o **readquire** ao acordar — por isso deve ser chamado com o mutex travado.

DISCIPLINAS DE SINALIZAÇÃO

SIGNAL AND CONTINUE

- Quem sinaliza **continua** dentro do monitor
- Quem foi acordado compete de novo na **fila de entrada**
- Comum em implementações práticas (ex.: Java, Go)

SIGNAL AND WAIT

- Quem foi acordado entra **imediatamente**
- Quem sinalizou vai para a **fila de entrada**
- Semântica próxima a Hoare — mais rara em linguagens mainstream

FILÓSOFOS COM MONITORES

- Um **monitor** `ForkMonitor` centraliza o estado
- Cada filósofo é um **processo** com variável de condição `OKtoEat[i]`
- `fork[i]` conta garfos disponíveis (0, 1 ou 2) para filósofo `i`
- **takeForks:** se `fork[i] ≠ 2`, espera em `OKtoEat[i]`; depois decrementa vizinhos
- **releaseForks:** incrementa vizinhos e sinaliza quem atingiu 2

> O filósofo só come quando `fork[i] = 2`; ao liberar, sinaliza vizinhos que atingiram 2 garfos.

Algorithm 7.5: Dining philosophers with a monitor

```
monitor ForkMonitor
   integer array[0..4] fork ← [2, . . . , 2]
   condition array[0..4] OKtoEat
   operation takeForks(integer i)
      if fork[i] ≠ 2
         waitC(OKtoEat[i])
      fork[i+1] ← fork[i+1] − 1
      fork[i−1] ← fork[i−1] − 1
   operation releaseForks(integer i)
      fork[i+1] ← fork[i+1] + 1
      fork[i−1] ← fork[i−1] + 1
      if fork[i+1] = 2
         signalC(OKtoEat[i+1])
      if fork[i−1] = 2
         signalC(OKtoEat[i−1])
```

philosopher i

```
    loop forever
p1:    think
p2:    takeForks(i)
p3:    eat
p4:    releaseForks(i)
```

Ben-Ari, Algorithm 7.5

FILÓSOFOS COM MONITORES EM GO

Tradução direta do Algoritmo 7.5 — contagem de garfos e variáveis de condição `OKtoEat`:

```go
type ForkMonitor struct {
  mu      sync.Mutex
  fork    [5]int
  OKtoEat [5]*sync.Cond
}

func NewForkMonitor() *ForkMonitor {
  m := &ForkMonitor{fork: [5]int{2, 2, 2, 2, 2}}
  for i := range m.OKtoEat {
    m.OKtoEat[i] = sync.NewCond(&m.mu)
  }
  return m
}

func (m *ForkMonitor) TakeForks(i int) {
```

MONITORES EM JAVA

Cada objeto Java possui um **lock intrínseco**. Métodos `synchronized` garantem exclusão mútua automática.

```
class Buffer {
  private int[] items;
  private int count = 0;

  // synchronized = exclusão mútua (monitor)
  public synchronized void put(int v) {
    while (count == items.length)
      wait();        // waitC — libera lock e espera
    items[count++] = v;
    notify();        // signalC — acorda um consumidor
  }

  public synchronized int take() {
    while (count == 0)
```

PRODUTOR-CONSUMIDOR COM MONITORES

Algorithm 7.3: producer–consumer (finite buffer, monitor)

```
monitor PC
   bufferType buffer ← empty
   condition notEmpty
   condition notFull

   operation append(datatype v)
      if buffer is full
         waitC(notFull)
      append(v, buffer)
      signalC(notEmpty)

   operation take()
      datatype w
      if buffer is empty
         waitC(notEmpty)
      w ← head(buffer)
      signalC(notFull)
      return w
```

producer	consumer
datatype d	datatype d
loop forever	loop forever
p1: d ← produce	q1: d ← PC.take
p2: PC.append(d)	q2: consume(d)

Ben-Ari, Algorithm 7.3

PRODUTOR-CONSUMIDOR COM MONITORES EM GO

Duas variáveis de condição: `notEmpty` e `notFull`, associadas ao mesmo mutex.

```go
type Buf struct {
  mu       sync.Mutex
  notEmpty *sync.Cond
  notFull  *sync.Cond
  items    []int
  limit    int
}

func NewBuf(limit int) *Buf {
  b := &Buf{limit: limit, items: make([]int, 0, limit)}
  b.notEmpty = sync.NewCond(&b.mu)
  b.notFull = sync.NewCond(&b.mu)
  return b
}
```

POR QUE `while` E NÃO `if`?

Com **signal and continue**, outra thread pode alterar o estado antes que a acordada execute — o predicado pode ter deixado de valer. Além disso, algumas APIs admitem **spurious wakeups**.

ERRADO

```
if len(b.items) == 0 {
    b.notEmpty.Wait()
}
// buffer pode estar vazio!
v := b.items[0]
```

CORRETO

```
for len(b.items) == 0 {
    b.notEmpty.Wait()
}
// loop garante que há itens
v := b.items[0]
```

PADRÃO COMPLETO: Take( ) COM for

```go
type Buf struct {
    mu       sync.Mutex
    notEmpty *sync.Cond
    notFull  *sync.Cond
    items    []int
    limit    int
}

func (b *Buf) Take() int {
    b.mu.Lock()
    defer b.mu.Unlock()

    for len(b.items) == 0 { // reavalia a cada despertar
        b.notEmpty.Wait()   // libera lock e espera
```

CANAIS PARA SINCRONIZAÇÃO

Comunicação como mecanismo de sincronização

CANAIS PARA SINCRONIZAÇÃO

Canais são um mecanismo de **troca de mensagens** (`send` / `recv`) que sincroniza processos sem compartilhar memória diretamente.

CANAL SÍNCRONO

- Sem buffer — envio bloqueia até o receptor estar pronto (**rendezvous**)
- Sincronização forte: emissor e receptor avançam juntos

CANAL ASSÍNCRONO

- Buffer de capacidade **N** desacopla emissor e receptor
- Envio bloqueia só quando **cheio**; recepção só quando **vazio**

> O bloqueio nas operações de envio e recepção transforma a **comunicação** em ponto de **sincronização** — o mesmo mecanismo coordena a troca de dados e a ordem de execução.

SEÇÃO CRÍTICA COM CANAIS

Canal booleano com buffer de tamanho 1 funciona como **semáforo binário**: receber adquire o token; enviar o devolve.

```go
mutex := make(chan bool, 1)
mutex <- true // token inicial

// goroutine qualquer:
<-mutex        // wait — adquire exclusão
// seção crítica
mutex <- true // signal — libera
```

Receber consome o token (bloqueia se o canal estiver vazio); enviar repõe. O **valor** (`true`) não importa — o que sincroniza é a **presença ou ausência** de um elemento no buffer.

VARIANTE IDIOMÁTICA EM GO: `chan struct{}`

Em Go, quando o canal serve apenas para **sinalização** (o valor transmitido é irrelevante), a convenção é usar `struct{}`.

```go
mutex := make(chan struct{}, 1)
mutex <- struct{}{} // token inicial

// goroutine qualquer:
<-mutex             // wait — adquire exclusão
// seção crítica
mutex <- struct{}{} // signal — libera
```

PRODUTOR-CONSUMIDOR COM CANAIS

- O buffer é um **canal bufferizado** de tamanho `N`
- Produtor e consumidor são **processos** independentes
- Envio bloqueia quando cheio; recepção bloqueia quando vazio

Algorithm 8.1: Producer-consumer (channels)	
channel of integer ch	
producer	**consumer**
integer x	integer y
loop forever	loop forever
p1: x ← produce	q1: ch ⇒ y
p2: ch ⇐ x	q2: consume(y)

Ben-Ari, Algorithm 8.1

O canal bufferizado concentra **buffer** e **sincronização** num único objeto: envio bloqueia quando cheio; recepção bloqueia quando vazio.

PRODUTOR-CONSUMIDOR EM GO (CANAIS)

```go
func producer(ch chan<- Item, produce func() Item) {
  for {
    ch <- produce()
  }
}

func consumer(ch <-chan Item, consume func(Item)) {
  for item := range ch {
    consume(item)
  }
}

func main() {
```

FILÓSOFOS COM CANAIS

- Cada garfo é um **canal booleano** com buffer 1
- Cada filósofo é um **processo**
- Adquirir garfo = **receber** do canal; liberar = **enviar** de volta

Algorithm 8.5: Dining philosophers with channels	
channel of boolean forks[5]	
philosopher i	**fork i**
boolean dummy	boolean dummy
loop forever	loop forever
p1: think	q1: forks[i] ⇐ true
p2: forks[i] ⇒ dummy	q2: forks[i] ⇒ dummy
p3: forks[i+1] ⇒ dummy	q3:
p4: eat	q4:
p5: forks[i] ⇐ true	q5:
p6: forks[i+1] ⇐ true	q6:

Ben-Ari, Algorithm 8.5

FILÓSOFOS EM GO (CANAIS)

```go
forks := make([]chan struct{}, 5)
for i := range forks {
  forks[i] = make(chan struct{}, 1)
  forks[i] <- struct{}{}
}

func philosopher(i int, eat func()) {
  left, right := forks[i], forks[(i+1)%5]
  <-left
  <-right
  eat()
  right <- struct{}{}
  left <- struct{}{}
```

COMPARAÇÃO DE MECANISMOS

Do hardware ao modelo de mensagens

TABELA COMPARATIVA

Mecanismo	Nível	Mem. compartilhada?	Exclusão mútua	Sinalização	Uso em Go
Semáforos	Baixo	Sim	Sim (binário)	Sim (geral)	Sem tipo nativo; canal bufferizado
Mutex	Baixo	Sim	Sim	Não	`sync.Mutex`
Monitores	Alto	Sim	Implícita	Cond vars	`sync.Mutex` + `sync.Cond`
Canais	Alto	Não necessário	Padrões com semântica de posse	Sim	`chan T`

PADRÕES EM SISTEMAS REAIS

SEÇÃO CRÍTICA

Contadores atômicos, registros de log, atualização de configuração compartilhada, qualquer operação lê-modifica-escreve em estado global.

PRODUTOR-CONSUMIDOR

Filas de mensagens (Kafka, RabbitMQ), pipelines de dados, filas de tarefas em servidores de aplicação e workers.

FILÓSOFOS (ALOCAÇÃO)

Pools de conexões, locks de linha ou tabela, qualquer conjunto de recursos onde processos competem por pares.

RESUMO

- **Semáforos**: P/V sobre contador atômico — Algoritmos 6.1, 6.8 e 6.10 (Ben-Ari)
- **Monitores**: encapsulamento + condition variables; `sync.Cond` em Go
- **Canais**: sincronização por comunicação — solução idiomática em Go para todos os problemas clássicos
- **Mesmos problemas, mecanismos diferentes** — a escolha depende do nível de abstração desejado e da linguagem